Type-Directed Discretization of Probabilistic Programs

KATHERINE WU, Cornell University, USA
JULES JACOBS, Jane Street, USA
KEVIN BATZ, Cornell University, USA
ALEXANDRA SILVA, Cornell University, USA

We study *exact discretization*as a , an intended semantics-preserving transformation for recursive, higher-order probabilistic programs with continuous distributions. We target programs where continuous values are compared against finitely many constants, so exact inference reduces to a discrete problem. Our central technical contribution is a non-local, type-directed analysis that infers where continuous values can be partitioned into finitely many observationally relevant regions, then rewrites sampling and comparison behavior over those regions. We call this transformation Slice. Because this construction is global and type-directed, correctness Correctness requires reasoning beyond the local syntax: we formalize the transformation and prove soundness for boolean queries using develop a coupling-style logical relations argument relation over operational semantics. As an application, transformed programs can be executed by discrete engines such as , , and The full semantic-preservation result remains conditional on an open weak-simulation obligation. The mathematical construction uses exact real interval masses; our prototype instead computes and normalizes floating-point CDF approximations, which discrete backends analyze as a numerical model. Our empirical evaluation shows two complementary strengths of when paired with discrete backends: it enables *exact inference* for that this implementation handles case studies combining continuous sampling, higher-order functions, and unbounded recursion , and, on benchmarks where direct comparison is possible, it is competitive with state-of-the-art exact inference systems for continuous programs.

CCS Concepts: • **TODO**;

Additional Key Words and Phrases: probabilistic programming, exact inference, continuous distributions, discretization, type systems

ACM Reference Format:
Katherine Wu, Jules Jacobs, Kevin Batz, and Alexandra Silva. 2026. Type-Directed Discretization of Probabilistic Programs. 1, 1 (August 2026), 122 pages. https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

1 Introduction

Probabilistic programming languages (PPLs) are widely used to model uncertainty [6, 10, 14, 15, 17, 21, 24, 25, 27, 34, 35, 37, 39–41, 45–47]. Programs with *continuous* distributions capture rich models but require reasoning about uncountable domains, whereas purely *discrete* programs often admit fast and precise inference but cannot express real-valued models.

Slice studies *exact discretization*: a semantics-preserving source-to-source transformation that converts, if possible, intended to preserve semantics while converting continuous programs into discrete ones. We target higher-order recursive programs with conditioning and continuous sampling, without arithmetic. The key mechanism is *type-directed cut inference*: our type system tracks each constant threshold (or *cut*) used in comparisons and uses this information to partition continuous domains into finitely many intervals. For example, $\texttt{uniform}(0,1) < 0.5$ discretizes to $\texttt{discrete}(0.5, 0.5) < 1$, yielding a discrete program that existing exact inference tools can process.

Cuts may arise far from their source expressions: a sampled value may flow through higher-order functions, conditionals, recursive calls, data structures, and observations before it is eventually compared. To discretize soundly, Slice propagates cut information along these non-local flows, collecting all observationally relevant

Authors' Contact Information: Katherine Wu, kaw324@cornell.edu, Cornell University, Ithaca, New York, USA; Jules Jacobs, julesjacobs@gmail.com, Jane Street, New York, New York, USA; Kevin Batz, ksb239@cornell.edu, Cornell University, Ithaca, New York, USA; Alexandra Silva, alexandra.silva@cornell.edu, Cornell University, Ithaca, New York, USA.

2026. ACM XXXX-XXXX/2026/8-ART
https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

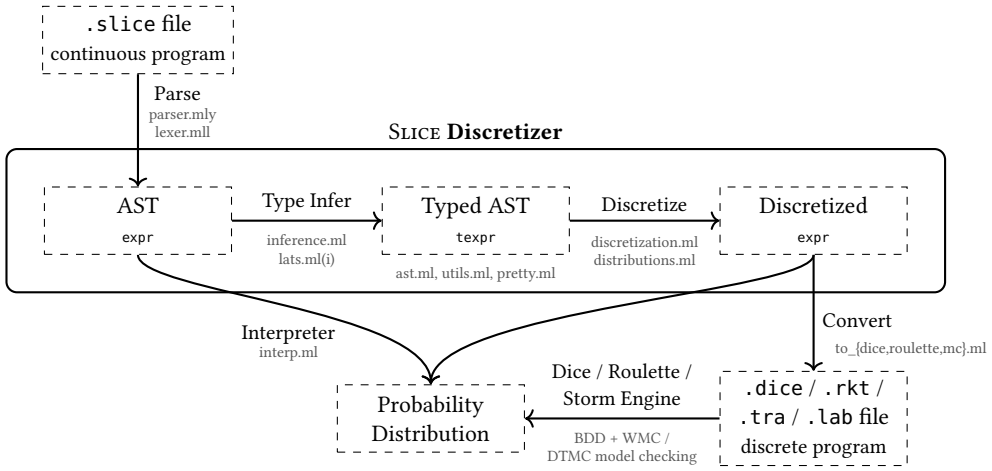


Fig. 1. Pipeline for exact discretization: typed Slice programs are transformed into semantically equivalent discrete programs, then evaluated by discrete enginesdiscretization. The intended mathematical transformation stage (type inference + discretization) is backend-agnosticuses exact interval masses; semantic preservation remains conditional on the open weak-simulation obligation. The artifact follows the same backend-agnostic architecture but supplies validated binary64 CDF approximations, so its backends analyze a numerical discrete model. Dice, Roulette, and Storm are examples of target backends.

cut points for each continuous value and using them to partition its domain into finitely many intervals (see §2 for illustrative examples). A type system provides a natural framework for this analysis: it gives a compositional account of how cut information propagates through each language construct (via one typing rule per langauge feature); it separates the analysis result, i.e. a type derivation, from the type inference algorithm used to infer it; and it supports our correctness argument, since semantic preservation can be stated as a logical relation indexed by our types carrying cut and value sets.

Our discretization method is subject to a principled restriction: a continuous value must ultimately be compared against constants; direct comparisons between two unknown reals such as $\texttt{uniform}(0,1) < \texttt{uniform}(0,1)$ cannot be discretized exactly and remain continuous. Nonetheless, benchmarks from the literature and our novel case studies show that many programs *can* be discretized by our system automatically.

Discretization itself is not new. Garg et al. [20] encode real-valued variables via user-chosen bit-precisions, trading accuracy for tractability. Other approximate discretization schemes exist in the literature [5, 12, 28]. In Slice, *exact discretization* means that the transformation is semantics-preserving: the discrete program preserves the relevant probabilities of the original continuous program. This differs from approximate discretization, which approximates the original semantics and may provide error bounds or convergence guarantees, but does not prove equality of source and transformed program probabilities in the sense of our main theoremnames the intended equality between the ideal exact-real transformation and the source semantics. The current proof establishes important fragments but leaves the general weak-simulation obligation open. The prototype is further separated from the ideal transformation because it computes interval masses with floating-point CDF routines and does not certify their error, as detailed in §7.

Figure 1 shows the concrete pipeline from typed source programs to discrete backends. Because discretization is source-to-source, any discrete PPL that supports finite distributions can serve as a backend; we instantiate this with Dice [27], Roulette [35], and Storm [16].

Outline. Starting from examples (§2) and concluding with related and future work in §8, we:

Type System (§3) Characterize when programs are discretizable.
Type Inference (§4) Infer types automatically to guide discretization.

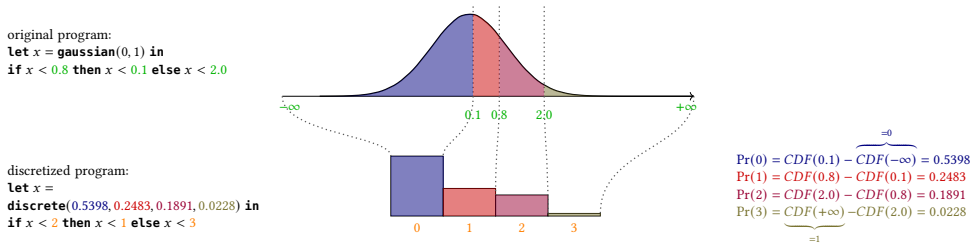


Fig. 2. Discretization of the original program into its semantically equivalent discretized program for example 2.1. The shaded regions show how intended exact-real target assigns the cuts at 0.1, 0.8, and 2.0 partition the continuous distribution into four intervals interval masses for $(-\infty, 0.1)$, $[0.1, 0.8)$, $[0.8, 2.0)$, and $[2.0, +\infty)$, whose masses become the . The displayed probabilities of outcomes 0, 1, 2, and 3, i.e., interval indices, in are rounded illustrations; the discrete distributionartifact computes validated binary64 CDF approximations. General semantic preservation remains conditional on the open weak-simulation obligation.

Exact Discretization (§5) Define a type-directed transformation from continuous programs to discrete ones using inferred type information.

Soundness (§6) Prove semantic preservation of the discretization transformation with Develop a coupling-style relational argument for semantic preservation, discharging a strict finite comparison fragment and isolating the remaining weak-simulation obligation [7, 13, 48].

Implementation, Case Studies, and Evaluation (§7) Implement Slice type inference and a numerical realization of the discretization transformation in OCaml, and provide Dice, Roulette, and Storm backends for running the discretized programs. We provide (i) case studies combining continuous sampling, higher-order functions, and unbounded recursion; to the best of our knowledge, there is no prior exact inference system that can cover no prior system covers programs *combining* all three of these features and (ii) case studies where Slice+Dice and Slice+Roulette show favorable scaling and empirical performance on new and existing benchmarks [22, 40].

2 Slice by Example

To build intuition for Slice, we start with simple examples demonstrating the key ideas, and then provide a more involved example containing higher-order recursion and lists in §2.6. Our pipeline performs *fully automatic exact inference* on automatically discretizes and analyzes all provided programs. The intended exact-real transformation uses exact interval masses, but its general semantic-equivalence result remains conditional on the open weak-simulation obligation. The current artifact computes those masses numerically, as discussed in §7.

2.1 Branching on a Single Continuous Variable

Consider a program that samples a real value from a standard Gaussian distribution and then performs two nested comparisons: first checking whether the sample is less than 0.8, then comparing it to either 0.1 or 2.0:

```
let x = gaussian(0,1) in if x < 0.8 then x < 0.1 else x < 2.0
```

We discretize this program by replacing the real-valued sample with a discrete interval index, as shown below and in Figure 2:

```
let x = discrete(0.5398, 0.2483, 0.1891, 0.0228) in if x < 2 then x < 1 else x < 3
```

Here, **discrete**(0.5398, 0.2483, 0.1891, 0.0228) already represents all four observationally distinguishable outcomes of the Gaussian sample. The precise real value of x never matters to the rest of the program; only the interval in which it falls does. This is the regime we target: continuous quantities observed only through finitely many comparison outcomes. *Slice's goal is to detect such situations automatically.* We partition the

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
respective (possibly overlapping) bounds *)
let init =
  (false, within 0.00 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 1.00) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Sample from a gaussian, then use
   the resulting value to update flags *)
let step = fun ys ->
  update (gaussian(0,1)) ys in

(* Number of samples determined by
   unbounded recursion *)
let cover = fix cover st :=
  if uniform(0,1) < 0.1 then st
  else cover (step st) in

(* Have all intervals been covered at
   least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run entire process on sampled intervals *)
let final = cover init in
  is_covered final
```

Fig. 4. An infinite stochastic process for random interval covering over $[0, 1]$.

from a gaussian (l. 31), and update the flag of every interval to true whenever y falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

The intended exact-real Slice *automatically* produces a *provably equivalent* (cf. §6) discrete program (transformation automatically produces the discrete program shown in §A.5). Even though this program still contains unbounded recursion, our back-end automatically determines that its stochastic execution behavior can be modeled as a *finite-state* Markov chain. Our key insight is that exact inference can then be performed via *probabilistic model checking*: we use . Its equivalence to this flagship source program remains conditional on the open weak-simulation obligation; the currently discharged strict finite comparison fragment does not cover the example's Gaussian sampling, lists, and recursion. The artifact nevertheless constructs a finite-state numerical model, and Storm [16] , which automatically computes that the sought-after probability is computes 0.034474... from its approximated transition weights. See §7.1 for details. To the best of our knowledge, our pipeline is the first capable of performing exact inference on suitable programs containing continuous sampling, higher-order functions, and unbounded recursion. In §7.1.2, we extend this program by conditioning *inside the recursion*.

3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

Lattice Solving. The lattice solver keeps track of the current contents of all B and V variables. Initially, all B and V variables are set to $\emptyset$. During the solving process, the variables may go up in the lattice to a finite set, or to $\top$. Each of the constraints is handled as follows:

- **Value containment** ($\mathsf{has}(V, c)$): We update V to include the constant c, i.e., $V := V \sqcup \{c\}$. If V is already $\top$, the constraint is trivially satisfied. (1)
- **Top assignment** ($V = \top$): We immediately set V to $\top$, representing that the value set contains all possible float values. (2)
- **Subset constraints** ($B_1 \sqsubseteq B_2$ or $V_1 \sqsubseteq V_2$): We ensure that the contents of the first variable are always a subset of the second. When B_1 or V_1 is updated, we propagate the changes to B_2 or V_2 by taking the least upper bound. (3)
- **Recoverability** ($\mathsf{recovers}(B, V)$): This constraint ensures that the cut set B can distinguish all values in V. If V is a finite set $\{v_1, ..., v_n\}$ with $n > 1$, we require that B contains at least $n - 1$ cuts to separate these values. If the cuts do not separate the values in the current solver state, we insert additional cuts to satisfy this condition. There is no canonical way to do this, so we use a heuristic: we insert a cut $<x$ for each value $x \in V$ that is not already separated by existing cuts. If $B = \top$, the constraint is trivially satisfied. (4)
- **Less-than answerability** ($\mathsf{answers}_<(B, V_1, V_2)$): Intuitively, this constraint ensures that B contains enough cuts to answer all possible comparisons of the form "$v_1 < v_2$" where $v_1 \in V_1$ and $v_2 \in V_2$. If $B = \top$, the constraint is trivially satisfied. Otherwise, we again have a heuristic choice: we add a cut $<v$ for each value $v \in V_2$ (if $V_2 \neq \top$) and $\leq v$ for each value $v \in V_1$ (if $V_1 \neq \top$). If V_1 and V_2 are both $\top$, we set B to $\top$. (5)
- **Less-equal answerability** ($\mathsf{answers}_\leq(B, V_1, V_2)$): Analogous to less-than. (5)
- **Equality constraints** ($B_1 = B_2$ or $V_1 = V_2$): We unify the variables, treating them as the same variable throughout the solving process. Any update to one variable is immediately reflected in all variables unified with it. (6)

The solver repeatedly processes constraints until reaching a fixed point, where no constraint causes any further updates to the variables. The monotonicity of the lattice operations ensures that this process terminates. This procedure may add more cuts to Bs than strictly necessary, but it never sets $B = \top$ unless that is the only way to satisfy all constraints. In other words, the discretization itself may be suboptimal in terms of the number of cuts.

Lattice constraint solving takes time polynomial in the size of the program: at worst, *every* constant in the program gets added to the cut set or value set of every subexpression of type $\texttt{float}[B; V]$.

Additional constraint for contextual equivalence. When the return type of the program involves a float type, either directly or as part of a larger compound type (e.g., $\texttt{float}[B; V] \to \texttt{float}[B'; V']$), we add an additional constraint to ensure that these float types are not discretized. We recursively traverse the return type and set all nested B sets to $\top$. This ensures that the observable behavior of the program is preserved after discretization, regardless of the context in which the program is used. When the return type is basic, such as **bool**, nothing needs to be done.

Example 4.2. To make the constraint solving process concrete, Figure 12 walks through §2.3. Each constraint obtained from the generation phase is processed by the solver, then collected to obtain the final cut and value sets.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression $e : \texttt{bool}$ and its type annotations, a *discretized* discretized expression $\mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool}$ such that e and $\mathcal{D}[\![e : \texttt{bool}]\!]$ are *semantically equivalent*, i.e., they induce that induces the same distribution. We refer to this semantics-preserving transformation as call this intended equality *exact discretization*, to distinguish it from approaches [20, 28]that approximate continuous programs via externally chosen precision parameters. Exactness here is the equality proved by Theorem 6.1: the transformed program preserves the relevant probabilities of the source program, not merely an approximation with an error bound. Intuitively, when a program uses continuous quantities only through finitely many comparison outcomes, exact inference should reduce to a discrete problem, and exact discretization makes

$e : \tau$	Condition	$\mathcal{D}[\![e : \tau]\!]$
$x : \tau$		x
c : **float**$[B; V]$	$B = \top$	c
	$B \neq \top$	$k_{\#(\lvert B \rvert+1)}$ such that $c \in \text{Intervals}(B)_k$
let $x = e_1 : \tau_1$ **in** $e_2 : \tau_2 : \tau_2$		**let** $x = \mathcal{D}[\![e_1 : \tau_1]\!]$ **in** $\mathcal{D}[\![e_2 : \tau_2]\!]$
(**if** e_1 : **bool then** $e_2 : \tau$ **else** $e_3 : \tau$) : τ		**if** $\mathcal{D}[\![e_1 : \textbf{bool}]\!]$ **then** $\mathcal{D}[\![e_2 : \tau]\!]$ **else** $\mathcal{D}[\![e_3 : \tau]\!]$
$(e_1 :$ **float**$[B; V_1] < e_2 :$ **float**$[B; V_2])$: **bool**	$B \neq \top$	$\mathcal{D}[\![e_1 : \textbf{float}[B; V_1]]\!] <_{\#(\lvert B \rvert+1)} \mathcal{D}[\![e_2 : \textbf{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \textbf{float}[B; V_1]]\!] < \mathcal{D}[\![e_2 : \textbf{float}[B; V_2]]\!]$
$(e_1 :$ **float**$[B; V_1] \leq e_2 :$ **float**$[B; V_2])$: **bool**	$B \neq \top$	$\mathcal{D}[\![e_1 : \textbf{float}[B; V_1]]\!] \leq_{\#(\lvert B \rvert+1)} \mathcal{D}[\![e_2 : \textbf{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \textbf{float}[B; V_1]]\!] \leq \mathcal{D}[\![e_2 : \textbf{float}[B; V_2]]\!]$
uniform$(e_1 :$ **float**$[B_1; V_1]$, $e_2 :$ **float**$[B_2; V_2])$: **float**$[B; V]$	$e_1, e_2 \in \mathbb{R},\ e_1 > e_2$	**diverge** : τ
	$B = \top$	**uniform**$(\mathcal{D}[\![e_1 : \textbf{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \textbf{float}[B_2; V_2]]\!])$
	$e_1, e_2 \in \mathbb{R}$, $B = \{\sim_1 b_1, \ldots, \sim_n b_n\}$	**discrete**$(p_0, \ldots, p_n)$ where $p_k = \text{CDF}(b_{k+1}) - \text{CDF}(b_k)$
	$B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, $V_1 = \{v_1, \ldots, v_n\}$, $V_2 = \{w_1, \ldots, w_m\}$	**let** $v_1' = \mathcal{D}[\![e_1 : \textbf{float}[B_1; V_1]]\!]$ **in** **let** $v_2' = \mathcal{D}[\![e_2 : \textbf{float}[B_2; V_2]]\!]$ **in** **if** $v_1' ==_{\#(\lvert B_1 \rvert+1)} \mathcal{D}[\![v_1 : \textbf{float}[B_1; V_1]]\!]$ && $v_2' ==_{\#(\lvert B_2 \rvert+1)} \mathcal{D}[\![w_1 : \textbf{float}[B_2; V_2]]\!]$ **then** $\mathcal{D}[\![\textbf{uniform}(v_1, w_1) : \textbf{float}[B; V]]\!]$ ... **else if** $v_1' ==_{\#(\lvert B_1 \rvert+1)} \mathcal{D}[\![v_i : \textbf{float}[B_1; V_1]]\!]$ && $v_2' ==_{\#(\lvert B_2 \rvert+1)} \mathcal{D}[\![w_j : \textbf{float}[B_2; V_2]]\!]$ **then** $\mathcal{D}[\![\textbf{uniform}(v_i, w_j) : \textbf{float}[B; V]]\!]$ ... **else** $\mathcal{D}[\![\textbf{uniform}(v_n, w_m) : \textbf{float}[B; V]]\!]$

Fig. 13. Discretization function, where $\text{CDF}(b_0) = 0$ and $\text{CDF}(b_{n+1}) = 1$. For the **uniform** distribution, $V = \top$ is implicit, and all remaining cut-set and value-set cases are ruled out by the constraints enforced during type inference. For all other language constructs, $\mathcal{D}[\![\cdot]\!]$ recurses structurally over them.

that reduction explicitexternally parameterized approximations [20, 28]. The general equality is conditional on the open weak-simulation obligation in §6; the local rules alone do not establish it. The artifact additionally realizes interval masses numerically rather than as exact reals.

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations; we focus on continuous uniform distributions, with others handled analogously. The formal rule and proof here cover the strict uniform family; the implementation exposes additional distribution-specific CDF paths that are not covered by this theorem. The only sub-expressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information; §6 proves that this non-local, type-mediated construction preserves semantics. §6 proves preservation for a strict finite comparison fragment and states the open weak-simulation obligation needed for the general result. Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form $e =$ **uniform**(e_1, e_2) of type **float**$[B; V]$ with $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$ can be *discretized* to an expression e' : **fin**$(n + 1)$ such that the probability that e evaluates to a value in $\text{Intervals}(B)_i$ *coincides* with the probability that e' evaluates to $i_{\#n+1}$. In particular, if *all* **float**$[B; V]$-typed subexpressions are finite-cut, then the given expression can be *fully discretized.*

Example 5.1. Consider a uniform distribution whose outcomes are compared to constants:

```
let x = uniform(0, 1) : float[B; T] in
```

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \texttt{fin}(|B|+1) & \text{if } \tau = \texttt{float}[B;V] \text{ and } B \neq \top \\ \texttt{float}[B;V] & \text{if } \tau = \texttt{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

Crucially, the *recoverability* constraints in the Continuous-rules from Figure 6 ensure that the parameter expressions of the respective sampling expressions can take only finitely many values, and that the cut sets are fine-grained enough to recover the values the parameter expressions evaluate to, as demonstrated in the preceding example.

A similar intuition applies to discretizable comparisons $e_1 < e_2$, where *both* expressions are non-constant but finite-cut. The *answers* constraints in the rule Less (resp. Less-Equal) ensure that at least one of e_1 and e_2 evaluates to only finitely many values, which enables discretization:

Example 5.3. Consider the following typed expression $(e_1 \leq e_2) : \texttt{bool}$, given by

```
(if discrete(0.5, 0.5) <#2 1#2 then
       uniform(0,10) : float[B; T] else 2.5 : float[B; {2.5}])
<=
(if discrete(0.5, 0.5) <#2 1#2
       then 2.5 : float[B; {2.5}] else 3.5 : float[B; {3.5}])
```

where $B = \{\leq 2.5, \leq 3.5\}$, and where e_1 : `float[B; T]` and e_2 : `float[B; {2.5,3.5}]`. Intuitively, e_1 samples from a mixture of a uniform and a dirac distribution, and e_2 samples from a finite distribution. Why can $e_1 \leq e_2$ be discretized even though neither e_1 nor e_2 is a constant? As suggested by its type, e_2 can take only finitely many values, namely $\{2.5, 3.5\}$. We apply $\mathcal{D}[\![\cdot]\!]$:

```
(if discrete(0.5, 0.5) <#2 1#2 then discrete(0.25, 0.1, 0.65) else 0#3)
<=
(if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3)
```

Notice that the probability of The displayed interval masses explain why the corresponding outcomes are paired: e_1 evaluating to some value in $(-\infty, 2.5]$ indeed *coincides* with the probability of the discretization of e_1 evaluating to $0_{\#3}$. For e_2, on the other hand, the probability of it evaluating assigns the stated mass to $(-\infty, 2.5]$, while e_2 assigns its point masses to 2.5 (resp. and 3.5) *coincides* with the probability of its discretization evaluating to $0_{\#3}$ (resp. $1_{\#3}$). For this reason, we obtain a semantically equivalent discretization of $e_1 \leq e_2$. The claim that the complete non-strict mixed comparison is semantically equivalent to the displayed target remains conditional on the open weak-simulation obligation.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

Theorem 5.4 (Discretization Is Well-Typed). *Let $\Gamma \vdash e : \tau$. We have $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, where $\mathcal{D}[\![\Gamma]\!]$ transforms all types in Γ, i.e., $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ for all $x \in \mathrm{dom}(\Gamma)$.*

6 Operational Semantics and Soundness

We now establish the soundness The intended soundness property of our approach in the following sense: For *every* is that for every expression $e : \texttt{bool}$ in the stated language fragment, possibly including *continuous sampling, higher-order functions, recursion, and conditioning,* continuous sampling, higher-order functions, recursion, and conditioning,

$e : \texttt{bool}$ *and* $\mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool}$ *(obtained from Figure 13) are semantically equivalent.*

The first step is to make formal sense of semantic equivalence. For that, we will define a monadic operational semantics for our language in §6.1. We will then proceed by proving the above claim. This is non-trivial: discretization The current proof discharges a strict finite comparison fragment. Extending the equality to the full stated language requires a weak-simulation argument; that obligation remains open. Discretization decisions are driven by non-local, type-propagated information, so we cannot proceed this missing argument cannot be replaced by a local rule-by-rule argument. To prove the claim rigorously, we relate the executions of e and $\mathcal{D}[\![e]\!]$ through a coupling-style logical relation in §6.1. proof.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle\colon \mathit{Expr}_\tau \to \mathcal{D}(\mathit{Vals}_\tau + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics has to distinguish between non-termination and observation failure, which we realize as follows: The "missing" probability mass in $\langle e : \tau\rangle$ is the probability that e diverges. The probability of $\bot$ is the probability of encountering an observation failure. With this model, semantic equivalence indeed captures our sought-after correctness claim: If $\langle e : \texttt{bool}\rangle = \langle \mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!]\colon \mathit{Expr}_\tau \to \mathcal{D}(\mathit{Expr}_\tau + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

Let $\tau = \texttt{bool}$ for the sake of concreteness. Care must be taken since, e.g., sub-expressions of type $\texttt{float}[B; \{3.5, 4.5\}]$ cannot evaluate to *arbitrary* reals but are restricted to the values $\{3.5, 4.5\}$. We tackle this by extracting from each type-annotated expression a suitable *skeleton*, where float constants are replaced by "holes" (hence the term "skeleton"). For instance, the expression

$$e \quad = \quad (x : \texttt{float}[\{<3.5, <4.5\}; \top] \;<\; y : \texttt{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \texttt{bool}$$

gives rise to the skeleton

$$s \quad = \quad (\square : \texttt{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \texttt{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \texttt{bool}$$

and the type annotations tell us that the possible values these holes can be filled with are in $\mathbb{R} \times \{3.5, 4.5\}$. We can thus inherit a measurable space over all well-typed expressions arising from this particular skeleton s from the standard Borel measurable space over $\mathbb{R} \times \{3.5, 4.5\}$. The measurable space over *all* well-typed expressions of type $\texttt{bool}$ (i.e., over $\mathit{Expr}_{\texttt{bool}}$) is then obtained from the measurable spaces of *all* skeletons that give rise to expressions of type $\texttt{bool}$.

Equipped with this family of measurable spaces, we define the small-step functional $[\![\cdot]\!]$ in the expected, type-preserving manner (cf. Tables 1 and 2). For instance, if e_1 is not a value, then[1]

$$[\![(\texttt{let}\ x = e_1 : \tau_1\ \texttt{in}\ e_2 : \tau_2) : \tau_2]\!] \;=\; \underbrace{[\![e_1 : \tau_1]\!] \gg\!= \lambda g. \begin{cases} \delta_\bot & \text{if } g = \bot \\ \delta_{(\texttt{let}\ x = g\ \texttt{in}\ e_2 : \tau_2) : \tau_2} & \text{otherwise} \end{cases}}_{\text{standard probabilistic monadic bind over suitable Markov kernels}},$$

i.e., $[\![e_1 : \tau_1]\!]$ yields the sub-distribution over expressions obtained from executing e_1 for *one step*, and the monadic bind $\gg\!=$ "folds" this distribution into the surrounding $\texttt{let}$-statement. The intuition for the remaining statements is analogous. Crucially, $\gg\!=$ propagates the probability of encountering $\bot$ — an observation failure — through executions. For instance, assume that e_1 from above is given by $\texttt{observe}\ (\text{false} : \texttt{bool}) : \texttt{unit}$. Since e_1 encounters an observation failure with probability 1, i.e., $[\![\texttt{observe}\ (\text{false} : \texttt{bool}) : \texttt{unit}]\!](\{\bot\}) = 1$, the same holds for the surrounding $\texttt{let}$-statement, i.e., also $[\![(\texttt{let}\ x = e_1 : \tau_1\ \texttt{in}\ e_2 : \tau_2) : \tau_2]\!](\{\bot\}) = 1$. All details — including proofs that all monadic binds are well-behaved since they operate on suitable Markov kernels — are provided in §F.4.

[1] Put formally, $(\mu \gg\!= f)(A) = \int f(x)(A) \cdot \mu(dx)$, see Definition F.28 for details. δ_e denotes the Dirac distribution of e.

6.1.2 From Small-Step to Big-Step Semantics. We now define $\langle\cdot\rangle$ via $[\![\cdot]\!]$. For that, we first lift the one-step semantics $[\![\cdot]\!]$ to an n-step semantics $\langle\cdot\rangle_n$ in the expected way:

$$\langle e:\tau\rangle_0 = \delta_{e:\tau} \qquad \text{and} \qquad \langle e:\tau\rangle_{n+1} \;=\; \langle e:\tau\rangle_n \gg\!= \lambda e'.\begin{cases}\delta_\perp & \text{if } e' = \perp\\ [\![e':\tau]\!] & \text{otherwise}\end{cases}$$

This construction is well-defined since the functional $[\![\cdot]\!]$ itself gives rise to a suitable Markov kernel (Lemma F.39). We then define for every type τ the functional $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \perp)$ as $\langle e:\tau\rangle = \lambda A.\lim_{n\to\infty}\langle e:\tau\rangle_n(A)$, which is well-defined by the monotone convergence theorem: the sequence $(\langle e:\tau\rangle_n)_{n\in\mathbb{N}}$ is monotonically increasing for every measurable set A over $Vals_\tau + \perp$ (cf. Lemma F.47) and upper-bounded by 1: executing a program for more and more steps can only increase the probability of terminating in a given set of values or an observation failure.

6.2 Soundness via Probabilistic Couplings

We first state the central theorem, then outline our coupling-style correctness argument.

Theorem 6.1 (Conditional soundness). *Let e : **bool**. Then Assume the weak-simulation obligation for the source and discretized operational semantics. For every e : **bool** in the stated fragment, $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$.*

Intuitively, we prove that the two programs can execute in a synchronized manner such Under the weak-simulation assumption, the intended argument synchronizes the two programs so that their probabilities of terminating in value v coincide for each $v \in \{\text{true}, \text{false}, \perp\}$. The currently proved strict finite comparison fragment supplies this synchronization directly.

Towards this end, define for every type τ the set $P_\tau = \{(e:\tau, \mathcal{D}[\![e:\tau]\!] : \mathcal{D}[\![\tau]\!]) \mid e:\tau\}$. We define a *coupled small-step semantics* $[\![\cdot,\cdot]\!]\colon P_\tau \to \mathcal{D}(P_\tau + \perp)$ (cf. Tables 3 and 4), which models the simultaneous execution of e and $\mathcal{D}[\![e:\tau]\!]$. The key property of this coupled semantics is that the semantics of both e and $\mathcal{D}[\![e:\tau]\!]$ can be recovered in a sense we formalize next.

Given $\mu \in \mathcal{D}(P_\tau + \perp)$, we define the *projection of μ onto the i-th component* for $i \in \{1,2\}$ as

$$\pi_i(\mu) = \lambda A.\int_{x\in P_\tau+\perp}\begin{cases}\delta_{e_i}(A) & x = (e_1,e_2)\\ \delta_\perp(A) & x = \perp\end{cases}\mu(dx)\,,$$

With projections, we recover the semantics of e and $\mathcal{D}[\![e:\tau]\!]$ Conditional on the weak-simulation obligation, projections recover the two semantics as follows:

Lemma 6.2 (Conditional projection recovery). *Let $e:\tau$. We have:*

(1) $\pi_1([\![e:\tau, \mathcal{D}[\![e:\tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![e:\tau]\!]$.
(2) $\pi_2([\![e:\tau, \mathcal{D}[\![e:\tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![\mathcal{D}[\![e:\tau]\!] : \mathcal{D}[\![\tau]\!]]\!]$.

Now, using Assuming the missing small-step recovery property, a limit construction analogous to §6.1.2 for obtaining a coupled *big-step* semantics$\langle\cdot,\cdot\rangle\colon P_\tau \to \mathcal{D}(V_\tau + \perp)$, where $V_\tau = \{(v:\tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau \text{ is a value}\}$, we prove a suitable continuity property of projection (cf. Lemma F.51) to conclude that Lemma 6.2 transfers to $\langle\cdot,\cdot\rangle$. Finally, observing yields a coupled big-step semantics. Continuity of projection then transfers the conditional recovery property to the limit. Observing that $V_{\mathbf{bool}} + \perp = \{(\text{true},\text{true}), (\text{false},\text{false}), \perp\}$ yields the desired conditional claim:

Proof Conditional proof of Theorem 6.1. Let Assume the weak-simulation obligation and let $v \in \{\text{true}, \text{false}, \bot\}$. We have

$$
\begin{aligned}
& \langle e : \mathbf{bool} \rangle(v) \\
= \; & \pi_1(\langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \rangle)(v) && \text{(Lemma 6.2.1 for } \langle \cdot, \cdot \rangle) \\
= \; & \pi_1(\langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \rangle)(v) && (P_{\mathbf{bool}} \text{ is discrete}) \\
= \; & \langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \rangle((\text{true}, \text{true})) \cdot \delta_{\text{true}}(v) \\
& + \langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \rangle((\text{false}, \text{false})) \cdot \delta_{\text{false}}(v) \\
& + \langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \rangle(\bot) \cdot \delta_{\bot}(v) && \text{(definition of } \pi_1\text{, argument is discrete)} \\
= \; & \pi_2(\langle e : \mathbf{bool}, \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \rangle)(v) && \text{(definition of } \pi_2\text{, argument is discrete)} \\
= \; & \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \rangle(v) \, . && \text{(Lemma 6.2.2 for } \langle \cdot, \cdot \rangle)
\end{aligned}
$$

7 Implementation, Case Studies, and Evaluation

We have implemented the Slice type inference and discretization procedures in OCaml. The implementation is open-source and available at https://github.com/[anonymized], together with all benchmarks, scripts, and a Docker image for reproduction. To compute the interval masses of continuous distributions, we use The intended formal transformation assigns each interval its exact real probability mass. The implementation instead subtracts binary64 values returned by the OCaml GSL package [19] for cumulative distribution function calculations. routines [19]. Non-finite, negative, or out-of-tolerance vectors are rejected; accepted vectors are normalized with a closed final endpoint. These steps produce a numerical probability vector, not a certified approximation to each ideal real mass. Dice and Roulette consume the emitted binary64 literals in their own weighted-inference pipelines, while Storm numerically analyzes the generated DTMC. None of these backend runs certifies equality with the continuous source semantics. The artifact uses neither symbolic exact masses nor certified interval or arbitrary-precision calculations and provides no end-to-end error bound. The full ideal semantic-equality result also remains conditional on an open weak-simulation obligation. All experiments were run in this Docker image on an Apple M3 with 16 GB of RAM.

This section is driven by two research questions:

(1) Does Slice enable exact inference on finite discrete analysis of programs combining continuous sampling, higher-order functions, and unbounded recursion?
(2) On programs that are within the scope of existing tools for exact inference exact inference tools, is the combination of Slice and discrete inference back-ends competitive?

For the first question, we provide a collection of case studies that involve continuous distributions together with recursion, higher-order functions, lists, or conditioning in §7.1. For these case studies, we use Slice to transform the original programs to finite discrete models and analyze the resulting program with Storm [16] — a probabilistic model checker.

For the second question, we evaluate Slice+Dice and Slice+Roulette on two classes of benchmarks in Sections 7.2 and 7.3: (i) synthetic scaling benchmarks that let us vary program size and dependency structure in a controlled way, and (ii) benchmark suites from prior work [22, 40]. We compare against Sppl [40], an exact inference engine supporting continuous probabilistic programs, on benchmarks that *both* Slice and Sppl support, and we additionally report Psi and Hakaru timings for the representative prior-work benchmarks. We remark, however, that Sppl supports arithmetic, as well as (soft) conditioning on measure zero events, which Slice does not support. Our evaluation merely serves the purpose of investigating investigates (i) the overhead of discretization and (ii) the benefits of being able to use using efficient *BDD-based* inference back-ends (i.e., Dice and Roulette) for exact inference on *continuous programs*on the numerical discrete models produced from continuous programs.

7.1 Slice+Storm: Case Studies

There is no exact inference system that, to our knowledge, can cover programs combining all three features: continuous sampling, higher-order functions, and unbounded recursion. As a consequence, there is no exact inference engine for the programs in Sections 2.6, 7.1.2 and 7.1.3. Instead, we use *probabilistic model checking*:

Continuous Program

```
1  (fix resample x :=
2    let _ = observe(x >= 0.2) in
3    if x <= 0.5 then true
4    else if x <= 0.8 then resample x
5    else
6      let y = uniform(0,1) in
7        resample y) 1
```

Discretized Program

```
9  (fix resample x :=
10   let _ = observe(x >= 1) in
11   if x <= 1 then true
12   else if x <= 2 then resample x
13   else
14     let y = discrete(0.2,0.3,0.3,0.2) in
15       resample y) 3
```

Markov Chain Construction & Model Checking

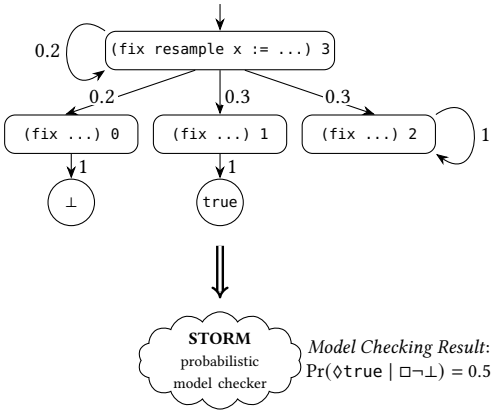


Fig. 15. Conditional resampling loop and its discretization. The continuous program is transformed into a finite-state discrete program (Slice alone takes less than 0.001s), which induces a finite Markov chain (generated in 0.001s) and can then be analyzed by Storm (with a model checking time of 0.024s). The ⊥-state in that Markov chain is entered when encountering an observation failure.

The small-step semantics of our language (cf. Tables 1 and 2) for discrete programs gives rise to a *Markov chain* modeling the stochastic process of program execution, inspired by [2] on imperative programs. If that Markov chain is finite, we can use the probabilistic model checker Storm [16] for exact inferencecomputes numerical reachability results for its supplied transition weights. Let us start with an illustrative example.

7.1.1 Conditional Resampling Loop. Consider the continuous program depicted in Figure 15, containing continuous sampling, unbounded recursion, and conditioning inside of the recursion. Slice automatically discretizes this program, also shown in Figure 15. We generate the corresponding The artifact generates a Markov chain by unfolding the its small-step semantics exporter in a breadth-first manner. In the presence of unbounded recursion, this Markov chain *might* be infinite and this process might thus diverge. If, however, it terminates, the resulting Markov chain is finite and precisely captures the execution behavior For this example generation terminates, producing a finite numerical model of the discretized program. For this example, this is the case: artifact. Even though the discretized program contains unbounded recursion, its execution behavior can be modelled as a generated model is finite-stateMarkov chain[2], where ⊥ is a special state entered when encountered encountering an observation failure.

Now, using the probabilistic model checker For the generated numerical Markov chain, Storm , we can perform exact inference: It is immediate that the Markov chain's probability $\Pr(\Diamond \texttt{true} \mid \Box\neg\bot)$ of eventually reaching computes reachability probabilities for the supplied transition weights. Observation-success conditioning and returned-value normalization are distinct in the presence of divergence. The former divides $\Pr(\Diamond \texttt{true})$ by $\Pr(\Box\neg\bot)$ and retains divergent paths. The normalized posterior over returned Booleans instead divides by $\Pr(\Diamond \texttt{true}) + \Pr(\Diamond \texttt{false})$, when this denominator is positive. For this example the raw masses are 0.375 for returning true(in LTL: ◊true) *given* that we never enter the observation-failure state ⊥ (in LTL: $\Box\neg\bot$) equals the measure of , 0.25 for observation failure, and 0.375 for divergence. Thus $\Pr(\Diamond \texttt{true} \mid \Box\neg\bot) = 0.5$, whereas the returned-value posterior assigns probability 1 to truein the program's normalized posterior distribution.

[2]We apply a straightforward optimization: Intermediate execution steps that do not branch probabilistically can be collapsed into a single transition. This is the reason why all states in Figure 15 correspond to the header of the recursive function.

7.1.2 Random Interval Covering of $[0, 1]$ *with Conditioning.* Our second case study extends the random interval covering example from §2.6 by conditioning inside of the recursion: We condition on *every* sample falling within *at most* one interval. The difference to the program in §2.6 is:

```
1  let non_overlap = fix non_overlap y :=
2    fun seen -> fun ys ->
3      match ys with
4      | nil -> true
5      | h :: t ->
6          let hit = (snd h) y in
7          if hit && seen then false
8          else non_overlap y (seen || hit) t
9      end in
11 let cover = fix cover st :=
12   if uniform(0,1) < 0.1 then st
13   else
14     let y = gaussian(0,1) in
15     let _ = observe(non_overlap y false st) in
16     cover (step y st)
17 in
18 let final = cover init in
19 is_covered final
```

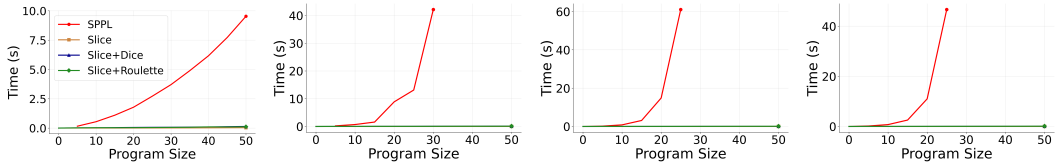
The key new component is the recursive function `non_overlap` (l. 1), which checks whether a sampled point y hits more than one interval of the initially sampled intervals. The loop `cover` (l. 11) repeatedly samples a Gaussian point and immediately conditions on this predicate via **`observe`** (l. 15), so only executions in which the point lies in at most one interval are retained. The measure of true in the conditional posterior distribution is thus the probability that this unbounded process hits every interval at least once *given* that each sampled point falls in at most one interval.

The program remains discretizable: each sampled point is still used only through comparisons against finitely many sampled interval boundaries, and `non_overlap` depends only on the resulting hit pattern, not on the exact real value of y. Slice replaces each Gaussian draw by its interval index, yielding a discrete program (shown in Figure 22) with a finite exact state space state space and numerically approximated transition weights. Slice takes 0.001s, and we generate a Markov chain consisting of 268878 states in 80 seconds. Storm takes 4.7 seconds for model checking, and reports that the sought-after probability is 0.000227....

Currently, our Markov chain construction process is rather naive: extending the number of sampling points or intervals yields the Markov chain to be infeasibly large. Developing efficient symbolic discrete inference engines for higher-order programs with unbounded recursion is not in the scope of this paper. However, our approach is an excellent motivation for this future direction.

7.1.3 Bloom Filter. We model a probabilistically populated variant of a Bloom filter [8], which is a data structure using hash functions for storing data space efficiently: one can insert data into a Bloom filter and subsequently query whether a given data point is stored in it. Due to aggressive compression via hashing, this process can be faulty: while it can never produce false-negatives, it can produce false-positives. Now conisder the following program (full program shown in Figure 23):

```
1  let insert = fun q -> fun h1 ->
2      fun h2 -> fun bf ->
3    let index1 = h1 q in
4    let index2 = h2 q in
5    let bf1 = set_true bf index1 in
6    set_true bf1 index2 in
7
8  let search = fun q -> fun h1 ->
9      fun h2 -> fun bf ->
10   let index1 = h1 q in
11   let index2 = h2 q in
12   (get bf index1) && (get bf index2) in
13
14 let h1 = fun q ->
15   if q <= -2 then 0
16   else if q <= 0 then 1
17   else if q <= 2 then 2
18   else 3 in
19 let h2 = fun q ->
20   if q <= -2 then 3
21   else if q <= 0 then 2
22   else if q <= 2 then 1
23   else 0 in
24
25 let init = (false, (false, (false, false))) in
26 let key = gaussian(0,1) in
27
28 let search_before_full = fix loop bf :=
29   if (full bf) then bf
30   else if (search key h1 h2 bf) then bf
31   else
32     let q = gaussian(0,1) in
33     loop (insert q h1 h2 bf) in
34
35 let bf_final = (search_before_full init) in
36   not (full bf_final)
```



(a) Cond. Ind. – Fork (b) Cond. Ind. – Rand. (c) Alt. Guard (d) Alt. Guard – Rand.

Fig. 16. Scaling behavior on representative benchmark families (additional families in §G.1.1). The plots emphasize asymptotic growth-rate differences and the resulting timeout frontier as program size increases. Time-out=120s; a disappearing line indicates timeout.

The program samples a Gaussian key (l. 26) and starts from an empty four-bit Bloom filter (l. 25). It then builds the filter recursively by drawing fresh Gaussian samples (l. 32) and inserting each one into the filter by hashing the sampled real twice using the comparison-based hash functions `h1` and `h2` (l. 14 and 19), following a locality-sensitive hashing technique that hashes similar input items into the same buckets. Each insertion sets the corresponding bits in the filter using `insert` (l. 1). The unbounded recursion continues until either the filter becomes full or the key is reported present by `search` (l. 8). The final Boolean result, `not (full bf_final)` (l. 36), therefore asks whether the key was found strictly before the filter became full.

The key simplification is that the hash functions never inspect the exact value of a sample. They only ask which region the sample falls into, using the cut points -2, 0, and 2. These cuts partition the real line into four regions, which already coincide with the Bloom filter's index space. therefore The intended ideal transformation replaces each Gaussian draw by its bucket indexwhile preserving exactly the probability of every collision pattern relevant to insertion and search. Preservation of every collision-pattern probability for this recursive example remains conditional on the open weak-simulation obligation; the artifact approximates the required Gaussian masses numerically. The fully discretized program is shown in Figure 24. Slice takes less than 0.001s, and we generate a Markov chain consisting of 14 states in 0.023 seconds. Storm takes 0.001 seconds for model checking, and reports the probability 0.91314....

7.2 Slice+Dice and Slice+Roulette: New Benchmarks for Scaling

In the absence of unbounded recursion, we can use Dice or Roulette as inference backends. We designed scalable benchmarks of conditionally independent continuous programs with varying variable dependencies. Formally, all programs in our benchmark can be described by the structure: $X_i \perp\!\!\!\perp \{ X_j : j < i, j \neq \pi(i) \} \mid X_{\pi(i)} \}$, where $\pi(i) \in \{1, \ldots, i-1\}$ is the parent index of X_i. Each variable X_i depends directly on only one earlier variable, namely its parent $X_{\pi(i)}$; conditioned on $X_{\pi(i)}$, it is independent of all other preceding variables. Detailed listings and graphs for variants of these benchmarks appear in the supplemental §G.1.1. Results are depicted in Figure 16.

We report runtime (seconds) for (i) Sppl, (ii) Slice (discretization only), (iii) Slice+Dice (discretization plus inference via Dice), and (iv) Slice+Roulette (discretization plus inference via Roulette). The timeout is 120 seconds. Program sizes are evaluated in increments of 5, from 0 to 50. For each size, we generate 10 random instances and report average runtime. We observe that discretization remains small across sizes, and takes less than 0.01 seconds for each program. End-to-end runtimes for Slice with discrete backends grow more mildly on these families; Sppl reaches a timeout frontier at larger instances. We conclude that using efficient BDD-based back-ends for exact inference on continuous programs via discretization on numerical discretizations can pay off for programs whose control flow induces strong conditional independence structure, allowing the resulting discrete models to remain compact even as program size grows.

7.3 Slice+Dice and Slice+Roulette: Existing Benchmarks

We next use benchmark suites from prior work, detailed in Tables 5 to 7 in Appendix §G.

The decision-tree benchmarks (Tables 5 and 6) from [1] assess fairness properties under different population models. DT_n denotes a tree with n conditionals, the second column specifies the population model, and the remaining columns report runtimes. We split Sppl execution into translation and fairness-judgment phases to

SPPL [40] emphasizes efficient exact symbolic inference for its fragment and supports features we do not currently automate, notably numeric transformations and conditioning on measure-zero events. λPSI [23] supports exact inference for higher-order continuous programs with nested inference and bounded recursion. Generating-function systems such as Genfer [51], Geni [33], and Prodigy [31] collectively support exact inference for infinite-support models and restricted unbounded-control models, but not general higher-order programs. PERPL [11] supports higher-order functions and unbounded recursion, but not continuous random variables. GuBPI [5] supports recursive higher-order programs with continuous distributions, but computes guaranteed converging bounds rather than exact posteriors. The intended ideal Slice targets the combination needed for our case studies: exact inference for programs that combine continuous sampling, higher-order functions, and unbounded recursiontransformation targets this combination through exact real interval masses, subject to the open weak-simulation obligation; the current artifact instead constructs uncertified numerical discrete models.

Relational proof techniques. Our soundness argument uses a coupling-style semantic relation between source and discretized executions. This connects directly to logical-relations and relational-semantics techniques for probabilistic programs [7, 13, 48], adapted here to a global, non-local, type-guided transformation. Notably, [7] considers only discrete probabilistic programs without conditioning, while [13, 48] support continuous distributions and scoring. More recently, [52] considered probabilistic programs with nested queries and recursion using a step-indexed model.

Future work. We would like to extend our work to support tractable symbolic integration for richer classes of continuous distributions (i.e. conjugate priors or other distributions admitting closed-form expressions), thus broadening the class of programs that Slice can discretize exactly. We also plan to investigate symbolic techniques à la Wang et al. [49] to speed-up inference in the presence of recursion.

```
let x = if flip() then 0.5 else 1.5 in gaussian(0, x) < 0.5
```

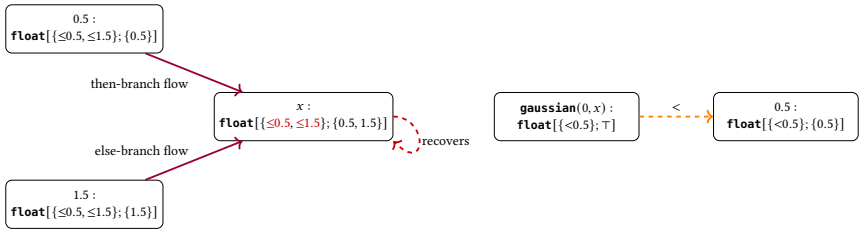


Fig. 19. Constraint graph populated with float type information. As before, solid arrows represent data flow, and dashed arrows represent comparison constraints. Using x as a Gaussian parameter induces a *recovers* constraint, the red dashed self-loop, which ensures a unique discrete index for each value of x.

```
      else
        if perfect <= 0 then
          (4.0 : float[{<10,<=10}; {4}])
        else
          (uniform(0, 4) : float[{<10,<=10}; T])
      : float[{<10,<=10}; T] in
let _ = observe(gpa : float[{<10,<=10}; T] <= 10 : float[{<10,<=10}; {10}]
          && gpa : float[{<10,<=10}; T] >= 10 : float[{<10,<=10}; {10}]) in
nationality > 0
```

The type system propagates the cuts <10 and ≤10 for all GPA expressions. These cuts induce the three regions:

$$(-\infty, 10), \qquad \{10\}, \qquad (10, \infty).$$

Under this partition, both continuous branches, `uniform(0,10)` and `uniform(0,4)`, place all of their probability mass in the first region. The constant 10.0 is mapped to the singleton region $\{10\}$, while the constant 4.0 is mapped to the region $(-\infty, 10)$. The resulting discretized program is therefore:

```
let nationality = discrete(0.5, 0.5) in
let perfect = discrete(0.01, 0.99) in
let gpa = if nationality <= 0 then
        if perfect <= 0 then 1
        else
          discrete(1.0, 0.0, 0.0)
      else
        if perfect <= 0 then 0
        else
          discrete(1.0, 0.0, 0.0) in
let _ = observe(gpa <= 1 && gpa >= 1)
nationality > 0
```

After discretization, the equality test `gpa = 10` becomes the discrete test that the GPA index is exactly the one corresponding to the singleton region $\{10\}$.

A.4 Point and Measure-Zero Observations

can condition In the ideal semantics, conditioning on an exact value is well-defined when that event has strictly positive probability mass, as in the Indian GPA example in §A.3. In that case, conditioning is well-defined and returns the correct answer. The exact set of programs supported by is therefore somewhat larger than one would initially expect, while not covering full point observations on continuous samples. The cut construction

retains that atom as a singleton cell. Equality between the full transformed program and the source remains conditional on the open weak-simulation obligation, and the binary64 artifact provides no numerical certificate.

The mechanism is the same as in §A.3: the cuts $<c$ and $\leq c$ induce the partition

$$(-\infty, c), \qquad \{c\}, \qquad (c, \infty),$$

so any probability mass assigned exactly to c is tracked as its own discrete outcome.

More precisely:

(1) Slice cannot handle programs where every control-flow path conditions on an event of measure zero, and therefore programs with exact observations on continuous samples are in general out of scope, i.e. the programs one typically writes in e.g. Stan are out of scope unless one widens exact conditioning to a small but finite interval.

(2) Slice can handle programs where at least one control-flow path conditions only on strictly-positive-probability events, even if that is a point observation, and even if other paths condition only on measure-zero events.

(3) It does not matter whether an event is measure-zero because it is point-conditioning on a continuous sample, or whether it is measure-zero for another reason, e.g. **observe**(**uniform**$(0,1) > 3$) or simply **observe**(false). The critical distinction for **observe**(E) is whether $\Pr(E) = 0$, not whether E is of the form "$x = 3.2$".

(4) will never give the wrong answer, even In the ideal extended semantics, when all paths condition on measure-zero events. It will give the empty subdistribution, indicating that , the result is the empty subdistribution: under a rejection-sampling interpretation, all samples are rejected with probability 1. Thus the user is notified, and never silently gives a wrong answerThis statement does not certify the binary64 artifact or discharge the open weak-simulation obligation.

For instance, this program:

```
let x = uniform(0,1) in
if flip() then
  let _ = observe(x >= 0.5 && x <= 0.5) in true
else
  let _ = observe(x <= 0.3 && x >= 0.3) in false
```

results in:

$$\Pr(\text{true}) = 0 \qquad \Pr(\text{false}) = 0.$$

Whereas this program:

```
let x = uniform(0,1) in
if flip() then
  let _ = observe(x >= 0.5 && x <= 0.5) in true
else
  let _ = observe(x <= 0.3001 && x >= 0.3) in false
```

results in:

$$\Pr(\text{true}) = 0 \qquad \Pr(\text{false}) = 1.$$

This is the correct behavior: consider repeatedly running the program under a rejection-sampling interpretation until an observation succeeds. With probability 1, the first successful test will be **observe**$(x \leq 0.3001 \;\&\&\; x \geq 0.3)$ and the program will therefore return false with probability 1.

A.5 Random Interval Covering Over $[0, 1]$

The discretized program for the case study from §2.6 is shown in Figure 20. Slice takes 0.000s, and we generate a Markov chain consisting of 192365 states in 38.69 seconds. Storm takes 0.69 seconds for model checking, and reports that the sought-after probability is 0.034474....